

AULA 3

DTOs e Validação de Dados

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso` (continuação da Aula 2)

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Entender o que é um DTO e por que ele não é uma entidade de banco
- Usar decorators do class-validator para declarar regras de validação
- Configurar o ValidationPipe globalmente na aplicação
- Entender o papel do class-transformer na conversão do payload

Até a Aula 2, o `POST /users` aceitava qualquer coisa no corpo da requisição — um número no lugar do nome, um e-mail sem `@`, ou nenhum campo. Esta aula resolve isso.

1. O que é um DTO

DTO é a sigla para *Data Transfer Object* — objeto de transferência de dados. É uma classe que define o formato dos dados que entram ou saem da API.

```
export class CreateUserDto {
  name: string;
  email: string;
}
```

DTO não é uma entidade de banco de dados. Ele representa o **contrato de comunicação** entre o cliente e o servidor, não como o dado fica guardado no banco. É comum, inclusive, que o formato de um DTO de criação seja diferente do formato da entidade salva — por exemplo, um DTO de cadastro pode ter um campo `password`, enquanto a entidade devolvida ao cliente nunca deveria expor esse campo (assunto retomado na Aula 9, de autenticação).

Por que uma classe, e não uma interface? Interfaces do TypeScript existem apenas em tempo de compilação — desaparecem completamente quando o código é transpilado para JavaScript. Os pipes do Nest, incluindo o `ValidationPipe`, precisam de informação que sobrevive em runtime, o que só uma classe oferece. Por isso a convenção do Nest é sempre usar classes para DTO.

Convenção de nomenclatura adotada no curso: cada operação tem seu próprio DTO — `CreateUserDto` para criação, `UpdateUserDto` para atualização (introduzido na Aula 4) — em vez de um DTO genérico tentando servir a tudo, já que as regras de obrigatoriedade mudam entre criar e atualizar.

2. Decorators de Validação (class-validator)

```
npm install class-validator class-transformer
```

```
export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;
}
```

2.1 Decorators mais comuns

Decorator	Regra
<code>@IsString()</code>	Deve ser uma string
<code>@IsInt()</code>	Deve ser um inteiro

@IsEmail()	Deve ter formato de e-mail válido
@IsOptional()	Campo não obrigatório
@IsNotEmpty()	Não pode chegar vazio
@MinLength() / @MaxLength()	Tamanho mínimo/máximo de texto
@Min() / @Max()	Intervalo de número
@IsUUID()	Deve ser um identificador único nesse formato

É possível combinar mais de um decorator na mesma propriedade — cada um adiciona uma regra, e todas precisam passar para a requisição ser aceita.

Importante: sozinhos, esses decorators não fazem nada além de anexar metadados à classe. Quem lê esses metadados e decide se o dado é válido é o `ValidationPipe`, apresentado a seguir. Sem essa peça, o Nest continua aceitando qualquer coisa no corpo da requisição.

3. ValidationPipe

O `ValidationPipe` é um pipe embutido do Nest que usa o `class-validator` para validar e o `class-transformer` para transformar o payload plano numa instância tipada do DTO. Configuração global, no `main.ts`:

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  } ),  
);
```

Opção	Efeito
<code>whitelist: true</code>	Remove do payload qualquer propriedade que não esteja no DTO
<code>forbidNonWhitelisted: true</code>	Rejeita com <code>400</code> se vier campo extra, em vez de apenas ignorá-lo
<code>transform: true</code>	Converte o payload plano em instância real do DTO, com os tipos corretos

Quando a validação falha, o Nest lança automaticamente uma exceção `400 Bad Request` com a lista de mensagens de erro de cada campo — sem necessidade de tratamento manual.

4. class-transformer

A função principal do `class-transformer`, usada internamente pelo `ValidationPipe`, é a `plainToInstance()`: ela recebe um objeto JavaScript puro — exatamente o que chega num POST — e o transforma numa instância real da classe DTO. Isso é necessário porque o `class-validator` precisa que o objeto seja uma instância real da classe decorada para enxergar os decorators aplicados.

Além da conversão de entrada, o `class-transformer` também ajuda a controlar o que sai na resposta — por exemplo, garantindo que um campo sensível, como uma senha, nunca seja devolvido no JSON de resposta. Esse uso retorna com mais detalhe na Aula 9, de autenticação.

Laboratório — Passo a Passo ## Passo 0 — Retomar o projeto

```
cd nest-api-curso  
npm run start:dev
```

Passo 1 — Instalar class-validator e class-transformer

```
npm install class-validator class-transformer
```

Passo 2 — Criar o DTO de criação

```
// src/users/dto/create-user.dto.ts
import { IsEmail, IsNotEmpty, IsString } from 'class-validator';

export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;
}
```

Passo 3 — Usar o DTO no Controller

```
@Post()
create(@Body() createUserDto: CreateUserDto) {
  return this.usersService.create(createUserDto);
}
```

Testar (ainda sem ValidationPipe): enviar `{ "name": 123, "email": "não-é-email" }` — a requisição ainda passa, provando que o DTO sozinho não valida nada.

Passo 4 — Ligar o ValidationPipe globalmente

```
// src/main.ts
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }),
);
```

Passo 5 — Testar a validação funcionando

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"","email":"não-é-email"}'
```

Esperado: status `400`, com mensagens de erro por campo.

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Ana","email":"ana@example.com","role":"admin"}'
```

Esperado: status `400` — o campo extra `role` derruba a requisição por causa do `forbidNonWhitelisted`.

Verificação de Conformidade

- ✔ DTO como classe (não interface) — conferido em docs.nestjs.com/controllers.
- ✔ Decorators `@IsString()`, `@IsNotEmpty()`, `@IsEmail()` — conferido na documentação do `class-validator` e em docs.nestjs.com/techniques/validation.
- ✔ `ValidationPipe` com `whitelist`, `forbidNonWhitelisted`, `transform` — conferido em docs.nestjs.com/techniques/validation.
- ✔ `class-transformer` e `plainToInstance()` usada internamente pelo `ValidationPipe` — conferido em docs.nestjs.com/pipes.

